

1: Start Spark with the JDBC jar

```
In [1]: from pyspark.sql import SparkSession
import os
```

```
jdbc_jar = os.path.expanduser("~/bigdata_lab/jars/mssql-jdbc.jar")
```

```
spark = SparkSession.builder \
    .appName("PersonA-Sources") \
    .config("spark.jars", jdbc_jar) \
    .getOrCreate()
```

```
print("Spark version:", spark.version)
```

WARNING: Using incubator modules: jdk.incubator.vector
 Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
 26/07/31 23:19:29 WARN Utils: Your hostname, Go, resolves to a loopback address: 127.0.1.1; using 10.255.255.254 instead (on interface lo)
 26/07/31 23:19:29 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
 26/07/31 23:19:30 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
 Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
 Setting default log level to "WARN".
 To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
 /home/ajith/bigdata_lab/venv/lib/python3.14/site-packages/pyspark/testing/utlis.py:127: FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some features may not work correctly. It is recommended to use pandas < 3.0.0 for now.
 require_minimum_pandas_version()

Spark version: 4.2.0

2: Load .env credentials

```
In [2]: from dotenv import load_dotenv
load_dotenv()
```

```
AZURE_JDBC_URL = (
    f"jdbc:sqlserver://{os.environ['AZURE_SQL_SERVER']}:1433;"
    f"database={os.environ['AZURE_SQL_DATABASE']};"
    "encrypt=true;trustServerCertificate=false;"
)
AZURE_USER = os.environ["AZURE_SQL_USERNAME"]
AZURE_PASSWORD = os.environ["AZURE_SQL_PASSWORD"]
```

3: Read movies + users from Azure SQL

```
In [3]: df_movies = spark.read.format("jdbc") \
    .option("url", AZURE_JDBC_URL) \
    .option("dbtable", "movies") \
```

```

.option("user", AZURE_USER) \
.option("password", AZURE_PASSWORD) \
.option("driver", "com.microsoft.sqlserver.jdbc.SQLServerDriver") \
.load()

df_movies.show(5)
print("Movies count:", df_movies.count())

df_users = spark.read.format("jdbc") \
.option("url", AZURE_JDBC_URL) \
.option("dbtable", "users") \
.option("user", AZURE_USER) \
.option("password", AZURE_PASSWORD) \
.option("driver", "com.microsoft.sqlserver.jdbc.SQLServerDriver") \
.load()

df_users.show(5)
print("Users count:", df_users.count())

```

```

+-----+-----+-----+-----+
|movie_id|          title|          genres|
+-----+-----+-----+
|      1| Toy Story (1995)|Animation|Childre...|
|      2|  Jumanji (1995)|Adventure|Childre...|
|      3|Grumpier Old Men ...|      Comedy|Romance|
|      4|Waiting to Exhale...|      Comedy|Drama|
|      5|Father of the Bri...|      Comedy|
+-----+-----+-----+
only showing top 5 rows
Movies count: 3883

+-----+-----+-----+-----+-----+
|user_id|gender|age|occupation|zip_code|
+-----+-----+-----+-----+-----+
|      1|  F|  1|      10|  48067|
|      2|  M| 56|      16|  70072|
|      3|  M| 25|      15|  55117|
|      4|  M| 45|       7|  02460|
|      5|  M| 25|      20|  55455|
+-----+-----+-----+-----+
only showing top 5 rows
Users count: 6040

```

4: Read the Hadoop CSV

```

In [4]: df_ratings_raw = spark.read.csv(
        os.path.expanduser("~/bigdata_lab/data/movielens/ratings_raw.csv"),
        header=True, inferSchema=True
    )
df_ratings_raw.show(5)
print("Ratings count:", df_ratings_raw.count())

```

```
+-----+-----+-----+-----+
|user_id|movie_id|rating|timestamp|
+-----+-----+-----+-----+
|      1|      1193|      5|978300760|
|      1|      661|      3|978302109|
|      1|      914|      3|978301968|
|      1|     3408|      4|978300275|
|      1|     2355|      5|978824291|
+-----+-----+-----+-----+
```

only showing top 5 rows

Ratings count: 1000209

5: Schema check

```
In [5]: df_movies.printSchema()
df_users.printSchema()
df_ratings_raw.printSchema()
```

```
root
|-- movie_id: integer (nullable = true)
|-- title: string (nullable = true)
|-- genres: string (nullable = true)
```

```
root
|-- user_id: integer (nullable = true)
|-- gender: string (nullable = true)
|-- age: integer (nullable = true)
|-- occupation: integer (nullable = true)
|-- zip_code: string (nullable = true)
```

```
root
|-- user_id: integer (nullable = true)
|-- movie_id: integer (nullable = true)
|-- rating: integer (nullable = true)
|-- timestamp: integer (nullable = true)
```

6. MongoDB imports + connection

```
In [6]: from dotenv import load_dotenv
load_dotenv()
```

Out[6]: True

```
In [7]: from pymongo import MongoClient
from pyspark.sql.types import StructType, StructField, IntegerType, ArrayType
from pyspark.sql.functions import explode, col

client = MongoClient(os.environ["MONGO_URI"])
collection = client["MovieLensDB"]["UserRatings"]
```

7: Pull documents, flatten nested ratings

```
In [8]: data = list(collection.find())

clean_data = [
    {
        "user_id": d["user_id"],
        "ratings": [
            {"movie_id": r["movie_id"], "rating": r["rating"], "timestamp": r["time
            for r in d.get("ratings", [])
        ],
    }
    for d in data
]

rating_schema = StructType([
    StructField("movie_id", IntegerType(), True),
    StructField("rating", IntegerType(), True),
    StructField("timestamp", IntegerType(), True),
])
user_schema = StructType([
    StructField("user_id", IntegerType(), True),
    StructField("ratings", ArrayType(rating_schema), True),
])

df_mongo = spark.createDataFrame(clean_data, schema=user_schema)

df_ratings_flat = df_mongo.withColumn("r", explode(col("ratings"))).select(
    "user_id", "r.movie_id", "r.rating", "r.timestamp"
)
```

9: Join everything on real keys

```
In [9]: df_final = df_ratings_flat.join(df_movies, "movie_id", "inner") \
        .join(df_users, "user_id", "inner")

df_final.show(10)
print("Final row count:", df_final.count())
```

26/07/31 23:21:40 WARN TaskSetManager: Stage 16 contains a task of very large size (1382 KiB). The maximum recommended task size is 1000 KiB.

```

+-----+-----+-----+-----+-----+-----+-----+-----+
---+-----+-----+
|user_id|movie_id|rating|timestamp|title|genres|gender|
age|occupation|zip_code|
+-----+-----+-----+-----+-----+-----+-----+-----+
---+-----+-----+
|      3|      1580|      3|978297663|Men in Black (1997)|Action|Adventure|...|      M|
25|      15|      55117|
|      4|      2366|      4|978294230|King Kong (1933)|Action|Adventure|...|      M|
45|      7|      02460|
|      5|      1580|      4|978245999|Men in Black (1997)|Action|Adventure|...|      M|
25|      20|      55455|
|      6|      1088|      5|978236670|Dirty Dancing (1987)|Musical|Romance|      F|
50|      9|      55117|
|      6|      1959|      3|978236612|Out of Africa (1985)|Drama|Romance|      F|
50|      9|      55117|
|      7|      1580|      4|978234842|Men in Black (1997)|Action|Adventure|...|      M|
35|      1|      06810|
|      8|      1580|      4|978229293|Men in Black (1997)|Action|Adventure|...|      M|
25|      12|      11413|
|     10|      1959|      5|978230330|Out of Africa (1985)|Drama|Romance|      F|
35|      1|      95370|
|     10|      3175|      5|979168132|Galaxy Quest (1999)|Adventure|Comedy|...|      F|
35|      1|      95370|
|     10|      1580|      5|978226401|Men in Black (1997)|Action|Adventure|...|      F|
35|      1|      95370|
+-----+-----+-----+-----+-----+-----+-----+-----+
---+-----+-----+
only showing top 10 rows

```

26/07/31 23:21:45 WARN TaskSetManager: Stage 25 contains a task of very large size (1382 KiB). The maximum recommended task size is 1000 KiB.

Final row count: 1000209

10: Aggregation

```

In [10]: df_final.createOrReplaceTempView("final")

spark.sql("""
    SELECT genres, ROUND(AVG(rating), 2) as avg_rating, COUNT(*) as num_ratings
    FROM final
    GROUP BY genres
    ORDER BY avg_rating DESC
""").show(20, truncate=False)

```

26/07/31 23:23:27 WARN TaskSetManager: Stage 37 contains a task of very large size (1382 KiB). The maximum recommended task size is 1000 KiB.

[Stage 43:=====> (1 + 7) / 8]

```

+-----+-----+-----+
|genres                                |avg_rating|num_ratings|
+-----+-----+-----+
|Animation|Comedy|Thriller                      |4.47      |688        | | |
|Sci-Fi|War                          |4.45      |1367       |
|Animation                                |4.39      |459        |
|Film-Noir|Mystery                      |4.37      |1584       |
|Adventure|War                      |4.35      |1644       |
|Action|Adventure|Drama|Sci-Fi|War    |4.29      |2990       |
|Film-Noir|Romance|Thriller          |4.29      |445        |
|Film-Noir|Sci-Fi                    |4.27      |1800       |
|Crime|Film-Noir                      |4.26      |867        |
|Film-Noir                                |4.26      |802        |
|Adventure|Children's|Drama|Musical  |4.25      |1718       |
|Action|Adventure|Romance|War        |4.25      |1057       |
|Drama|Film-Noir                      |4.22      |628        |
|Film-Noir|Thriller                  |4.21      |1983       |
|Crime|Film-Noir|Mystery              |4.2       |292        |
|Comedy|Mystery|Romance|Thriller      |4.18      |505        |
|Comedy|Drama|Musical                 |4.18      |1118       |
|Comedy|Mystery|Thriller               |4.17      |672        |
|Action|Adventure|Animation           |4.15      |345        |
|Comedy|Drama|Western                 |4.14      |807        |
+-----+-----+-----+
only showing top 20 rows

```

11: Export

```
In [11]: df_final.toPandas().to_csv("final_output.csv", index=False)
print("Saved final_output.csv")
```

```

/home/ajith/bigdata_lab/venv/lib/python3.14/site-packages/pyspark/sql/pandas/convers
ion.py:298: FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some
features may not work correctly. It is recommended to use pandas < 3.0.0 for now.
    require_minimum_pandas_version()
/home/ajith/bigdata_lab/venv/lib/python3.14/site-packages/pyspark/sql/pandas/convers
ion.py:348: UserWarning: toPandas attempted Arrow optimization because 'spark.sql.ex
ecution.arrow.pyspark.enabled' is set to true; however, failed by the reason below:
    [PACKAGE_NOT_INSTALLED] PyArrow >= 18.0.0 must be installed; however, it was not f
ound.
Attempting non-optimization as 'spark.sql.execution.arrow.pyspark.fallback.enabled'
is set to true.
    warn(msg)
26/07/31 23:24:04 WARN TaskSetManager: Stage 49 contains a task of very large size
(1382 KiB). The maximum recommended task size is 1000 KiB.

```

Saved final_output.csv

Build 2 NotebookLM files

1. Benchmark file: aggregated, small, covers all genres/occupations (This groups every row by genre + occupation combination, and calculates the average rating and count for

each pairing.)

```
In [12]: spark.sql("""
        SELECT genres, occupation, ROUND(AVG(rating), 2) as avg_rating, COUNT(*) as num
        FROM final
        GROUP BY genres, occupation
        """).toPandas().to_csv("genre_occupation_benchmark.csv", index=False)

print("Saved genre_occupation_benchmark.csv")
```

/home/ajith/bigdata_lab/venv/lib/python3.14/site-packages/pyspark/sql/pandas/conversion.py:298: FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some features may not work correctly. It is recommended to use pandas < 3.0.0 for now.
 require_minimum_pandas_version()
/home/ajith/bigdata_lab/venv/lib/python3.14/site-packages/pyspark/sql/pandas/conversion.py:348: UserWarning: toPandas attempted Arrow optimization because 'spark.sql.execution.arrow.pyspark.enabled' is set to true; however, failed by the reason below:
 [PACKAGE_NOT_INSTALLED] PyArrow >= 18.0.0 must be installed; however, it was not found.
 Attempting non-optimization as 'spark.sql.execution.arrow.pyspark.fallback.enabled' is set to true.
 warn(msg)
26/07/31 23:34:56 WARN TaskSetManager: Stage 56 contains a task of very large size (1382 KiB). The maximum recommended task size is 1000 KiB.

Saved genre_occupation_benchmark.csv

Build 2 NotebookLM files

2. Row-level subset — check the count before exporting

```
In [13]: subset = df_final.filter(
        df_final.genres.contains("Horror") | df_final.genres.contains("Documentary")
    )
print("Subset row count:", subset.count())
```

26/07/31 23:37:11 WARN TaskSetManager: Stage 70 contains a task of very large size (1382 KiB). The maximum recommended task size is 1000 KiB.

Subset row count: 84296

```
In [14]: df_final.filter(df_final.genres.contains("Horror")).count()
```

26/07/31 23:38:18 WARN TaskSetManager: Stage 82 contains a task of very large size (1382 KiB). The maximum recommended task size is 1000 KiB.

Out[14]: 76386

```
In [15]: df_final.filter(df_final.genres.contains("Documentary")).count()
```

26/07/31 23:38:32 WARN TaskSetManager: Stage 94 contains a task of very large size (1382 KiB). The maximum recommended task size is 1000 KiB.

Out[15]: 7910

```
In [16]: df_final.filter(df_final.genres.contains("Film-Noir")).count()
```

26/07/31 23:39:55 WARN TaskSetManager: Stage 106 contains a task of very large size (1382 KiB). The maximum recommended task size is 1000 KiB.

Out[16]: 18261

```
In [17]: df_final.filter(df_final.genres.contains("Western")).count()
```

26/07/31 23:40:09 WARN TaskSetManager: Stage 118 contains a task of very large size (1382 KiB). The maximum recommended task size is 1000 KiB.

Out[17]: 20683

Building a NotebookLM-Ready Subset

NotebookLM enforces a hard limit of 10,000 rows per uploaded source. Our full joined dataset (`final_output.csv`) has 1,000,209 rows, so it cannot be uploaded directly.

We initially tried scoping the subset by genre (e.g. Horror vs. Documentary) to give NotebookLM a meaningful contrast to reason about. However, every genre pairing we checked was either far too large on its own (Horror: ~76k, Film-Noir/Western: 18k+) or would have required an arbitrary secondary filter (e.g. by year or rating) with no clear analytical justification.

Instead, we use a random sample (`fraction=0.008` , `seed=42` for reproducibility) of the full joined dataset, yielding ~8,000 rows. Unlike a genre-scoped subset, a random sample is statistically representative of the whole dataset, making it well suited to prompts about general rating behavior and patterns, rather than a specific category contrast.

This subset is used alongside `genre_occupation_benchmark.csv` (a full, aggregated summary across all genres and occupations, well under the row limit on its own) together they give NotebookLM both the big-picture view and real row-level detail to work from.

The fixed `seed=42` means this exact sample can be regenerated identically by anyone re-running this notebook; the row count and contents will match every time, rather than producing a different random subset on each run.

```
In [18]: sample = df_final.sample(fraction=0.008, seed=42)
print("Sample row count:", sample.count())
```

26/07/31 23:43:17 WARN TaskSetManager: Stage 130 contains a task of very large size (1382 KiB). The maximum recommended task size is 1000 KiB.

Sample row count: 8033

```
In [19]: sample.toPandas().to_csv("ratings_sample_subset.csv", index=False)
print("Saved sample subset")
```

```
/home/ajith/bigdata_lab/venv/lib/python3.14/site-packages/pyspark/sql/pandas/conversion.py:298: FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some features may not work correctly. It is recommended to use pandas < 3.0.0 for now.
  require_minimum_pandas_version()
/home/ajith/bigdata_lab/venv/lib/python3.14/site-packages/pyspark/sql/pandas/conversion.py:348: UserWarning: toPandas attempted Arrow optimization because 'spark.sql.execution.arrow.pyspark.enabled' is set to true; however, failed by the reason below:
  [PACKAGE_NOT_INSTALLED] PyArrow >= 18.0.0 must be installed; however, it was not found.
Attempting non-optimization as 'spark.sql.execution.arrow.pyspark.fallback.enabled' is set to true.
  warn(msg)
26/07/31 23:43:33 WARN TaskSetManager: Stage 142 contains a task of very large size (1382 KiB). The maximum recommended task size is 1000 KiB.
```

Saved sample subset

Repository Contents & What's Excluded

This repository contains the code, notebook, and small derived output files needed to reproduce and review this project. A few large files are deliberately **not** committed to git, for the same reason it's standard practice in most data projects: they're large, easily regenerable, and don't belong in version control history.

Included:

- `*.ipynb` : The full Jupyter notebook(s) showing the pipeline end to end: loading Azure SQL, Hadoop CSV, and MongoDB Atlas, joining them in Spark, and building the final analysis-ready outputs
- `azure_sql_loader.py`, `load_movielens_azuresql.py`, `load_movielens_mongodb.py`, `ddl_movielens_schema.sql` - the source loading scripts
- `genre_occupation_benchmark.csv` - a small, aggregated summary (grouped by genre and occupation) covering the entire dataset
- `ratings_sample_subset.csv` - an ~8,000-row random sample of the full joined dataset (seed=42, reproducible), sized to fit within NotebookLM's 10,000-row upload limit

Excluded (not committed):

- `final_output.csv` (73MB, 1,000,209 rows) - the full joined dataset produced by the Spark pipeline. Not committed due to size; it can be regenerated in full by re-running the notebook end to end
- `data/movielens/ratings_raw.csv` and the raw `ml-1m/` dataset files - publicly downloadable from GroupLens (<https://grouplens.org/datasets/movielens/>),

regenerated automatically by the notebook's download step rather than stored in the repo

- `.env` - contains database credentials, excluded for security and never committed at any point in this project's history

Anyone cloning this repository can fully reproduce every excluded file by running the notebook top to bottom with their own `.env` credentials configured.